MercadoJá — do modelo ao desempenho

Trabalho Prático — Banco de Dados II

SGBD e versão usada nas medições: PostgreSQL 18 (Neon, ambiente de produção do aluno) para as medições principais; PostgreSQL 16 (ambiente de validação) para itens onde uma limitação de armazenamento do plano gratuito do Neon impediu a medição ao vivo (ver seção de Limitações).

Volume de dados: escala completa do contrato — 100.000 clientes, 5.000 lojas, 40.000 produtos, 600.000 pedidos, 1.579.193 itens de pedido, 492.273 pagamentos, 432.122 entregas, 250.000 avaliações.

1. Modelo e decisões de normalização

O esquema tem 9 tabelas em 3FN: cliente, loja, categoria (auto-relacionada, representando a hierarquia raiz/filha em 2 níveis), produto, pedido, item_pedido (tabela associativa N:N entre pedido e produto, carregando quantidade e desconto_item — atributos da própria associação, não de pedido nem de produto isoladamente), pagamento, entrega e avaliação.

Decisões de normalização

- Cliente separado de Pedido: nome, e-mail e cidade do cliente dependem só de cliente_id, não da chave do pedido. Mantê-los junto ao pedido geraria anomalia de atualização.
- Produto separado de Loja e Categoria: preço e estoque dependem de produto_id, não do pedido. item_pedido guarda preco_unitario como cópia intencional (histórico do preço no momento da venda — um fato sobre a venda, não sobre o produto atual).
- Categoria com auto-relacionamento (categoria_pai_id) em vez de duas tabelas separadas para raiz e filha — modela corretamente uma hierarquia de 2 níveis sem duplicar estrutura de colunas.
- item_pedido como tabela associativa: a relação pedido ↔ produto é N:N e carrega atributos próprios (quantidade, desconto_item) — caso clássico que exige tabela associativa em 3FN.
- Pagamento e Entrega como tabelas separadas de Pedido: são eventos com ciclo de vida próprio (podem não existir ainda, podem ter múltiplas tentativas) e não fazem sentido como colunas do pedido.

Denormalização intencional (e por quê)

pedido.valor_produtos, valor_frete e valor_desconto são deriváveis de item_pedido, mas foram mantidos como colunas do cabeçalho porque (a) o contrato de dados do enunciado pede esses campos explicitamente no pedido e (b) as consultas analíticas mais pesadas (Q1, Q4) agregam por esse valor em praticamente toda leitura — recalcular via SUM sobre 1,58 milhão de linhas de item_pedido a cada consulta do painel comercial seria caro demais para compensar a pureza normalizada. O custo é a responsabilidade de manter os dois lados consistentes na escrita (aplicação ou trigger) — decisão de projeto consciente, documentada aqui, não um erro de modelagem.

2. Consultas de negócio (Q1–Q9)

As nove consultas estão em sql/q1.sql a sql/q9.sql, cada uma comentada. Destaques de implementação:

- Q1/Q2/Q8 usam COALESCE(categoria_pai_id, categoria_id) para achar a categoria raiz de qualquer produto, funcionando tanto para produtos ligados direto a uma raiz quanto a uma folha.
- Q3 trata "gasto total" como valor_produtos − valor_desconto + valor_frete, ignorando pedidos cancelados tanto no total quanto na data do último pedido.
- Q5 usa ILIKE '%fone bluetooth%', que não é sargável em B-tree — por isso criamos um índice GIN de trigramas (pg_trgm) especificamente para ela.
- Q6 interpreta "pedidos entregues no trimestre" pela data de entrega (entrega.data_entrega), não pela data do pedido, pois a pergunta é sobre performance de entrega.
- Q8 resolve com ROW_NUMBER() OVER (PARTITION BY categoria_raiz ORDER BY unidades DESC, produto_id).

Achado importante durante o desenvolvimento: todas as consultas com LIMIT ou ROW_NUMBER precisaram de um desempate explícito (ex.: ORDER BY unidades DESC, produto_id). Sem isso, descobrimos que o corte do LIMIT entre linhas empatadas não é determinístico — duas execuções da mesma consulta podiam devolver conjuntos de linhas diferentes, o que violaria a exigência do enunciado de resultados corretos e reproduzíveis. Corrigimos Q2, Q3, Q5, Q6, Q7 e Q8 com uma coluna de desempate estável (geralmente o id) antes de fechar os resultados.

3. Consulta legada — diagnóstico e reescrita

Por que a consulta original não usa índice sobre a data

to_char(p.data_pedido, 'YYYY-MM') IN (...) aplica uma função sobre cada valor de data_pedido antes de comparar. Um índice B-tree é ordenado pelo valor cru da coluna, não pelo texto que a função produziria — o planejador não consegue provar que o resultado da função cai num intervalo contíguo do índice, então descarta o índice e faz Seq Scan completo. É o mesmo efeito descrito no enunciado: "função em cima da coluna mata o índice".

Reescrita

Trocamos o filtro por um intervalo direto de timestamp (data_pedido >= '2026-03-01' AND data_pedido < '2026-07-01'), sargável, e o OR de igualdades em uf_entrega por um IN equivalente. Confirmamos por comparação de arquivo (Compare-Object no PowerShell) que os dois SQLs devolvem exatamente as mesmas linhas com os mesmos valores — sem nenhuma diferença.

Resultado medido no ambiente do aluno (Neon, 3 execuções, mediana)

	Antes	Depois	Ganho
Tempo (mediano)	414,8 ms	133,2 ms	67,9%
Blocos lidos	1.368	27	98,0%
Nó principal	Seq Scan (varredura completa)	Index Only Scan (idx_pedido_status_data_uf)	—

4. Antes × depois das 9 consultas (evidência própria, Neon)

Tabela consolidada (3 execuções cada, mediana — ver evidencias/RESUMO_GERAL.md e evidencias/qN/ para os planos brutos):

Consulta	Antes (ms)	Depois (ms)	Ganho	Blocos antes	Blocos depois
Q1 — Relatório da diretoria	2332,2	2292,2	1,7%	1.351	6.286
Q2 — Top produtos	2162,2	1940,3	10,3%	3.776	8.062
Q3 — Reengajamento	916,9	917,7	-0,1%	1.741	597.437
Q4 — Painel comercial	216,3	197,8	8,6%	2.529	7.576
Q5 — Busca no catálogo	12,9	1,2	90,7%	658	832
Q6 — Lojas de alta performance	692,0	630,1	8,9%	6.437	14.732
Q7 — Repetição de pedidos	707,2	655,6	7,3%	4.719	8.935
Q8 — Top 3 por categoria	1698,2	1727,9	-1,7%	8.264	12.740
Q9 (bônus) — Coorte de retenção	1819,6	1467,2	19,4%	219	15.086
Legada	414,8	133,2	67,9%	1.368	27

Interpretação honesta (não só os números que "deram certo")

Onde o índice ajudou muito (Q5, legada): consultas cujo filtro é bastante seletivo — Q5 procura ~440 produtos entre 40.000 (~1%); a legada filtra 4 meses de 30 e 4 UFs. Nesses casos, um Index (Only) Scan evita ler a tabela inteira.

Onde o índice quase não mudou nada (Q1, Q3, Q8, e ganho pequeno em Q2/Q4/Q6/Q7): essas consultas agregam boa parte da tabela item_pedido (1,58 milhão de linhas) dentro de uma janela de 12 meses que cobre a maior parte do histórico de 30 meses da operação. Bate com a dica do próprio enunciado: "quando a consulta agrega

tudo, o custo está na leitura, não na busca" — não existe índice que reduza o custo de ler e agregar a maioria das linhas de uma tabela grande.

Achado que exigiu investigação (Q3 — blocos "explodiram" de 1.741 para 597.437 mesmo com tempo estável): conferimos a linha "Buffers:" do plano com índice e encontramos shared hit=603.618 e nenhum shared read — ou seja, 100% dos blocos tocados foram acerto de cache, zero leitura de disco nova. O plano com índice passou a usar um Nested Loop que revisita repetidamente páginas já residentes em memória — caro em contagem de blocos, mas sem custo real de I/O, o que explica por que o tempo de execução não piorou. Isso mostra o limite do "blocos lidos" como proxy único de custo: quando tempo e blocos discordam, vale abrir o plano e olhar shared hit vs shared read, não só o total.

5. Análises A1–A6

A1 — Antes/depois por consulta

Ver evidencias/qN/ (uma pasta por consulta: antes.txt, depois.txt, resumo.md) e evidencias/legado/ para a consulta legada. Todos os planos brutos vêm de EXPLAIN (ANALYZE, BUFFERS) rodado 3 vezes no banco Neon do aluno.

A2 — Justificativa de cada índice

Ver os comentários em sql/indices.sql — cada CREATE INDEX tem, logo acima, qual(is) consulta(s) atende e por quê (seletividade, coluna de junção, ordenação, cobertura). Resumo dos 13 índices criados:

Índice	Tipo	Atende
idx_pedido_status_data_uf	B-tree (status, data_pedido, uf_entrega)	Q1, Q2, Q4, Q8, legada
idx_pedido_cliente_data	B-tree (cliente_id, data_pedido)	Q3, Q7, Q9
idx_item_pedido_produto	B-tree (produto_id)	Q1, Q2, Q8
idx_item_pedido_pedido	B-tree (pedido_id)	Junções item_pedido ↔ pedido
idx_produto_categoria	B-tree (categoria_id)	Q1, Q2, Q8
idx_produto_loja	B-tree (loja_id)	Q2, Q6
idx_produto_nome_trgm	GIN trigramas (pg_trgm)	Q5 (ILIKE '%...%')
idx_produto_ativo_preco	B-tree parcial (WHERE ativo)	Q5
idx_entrega_status_data	B-tree (status, data_entrega)	Q6
idx_entrega_pedido	B-tree (pedido_id)	Junções entrega ↔ pedido
idx_avaliacao_produto / _pedido	B-tree	Q6
idx_pagamento_pedido	B-tree (pedido_id)	Integridade referencial / DELETE
idx_pedido_canal_ignorado	B-tree (canal_venda)	Contraprova A4 (deliberadamente pouco útil)

A3 — Reescrita sem criar índice

Reescrevemos a Q7 "agregando antes de juntar": a versão original junta pedido com cliente para as 600.000 linhas de pedido e só agrupa depois; a versão alternativa (sql/a3_q7_agregar_antes.sql) agrupa pedido sozinho primeiro (o resultado já sai pequeno — só combinações cliente+dia com 2+ pedidos) e só então junta com cliente. Resultado idêntico (conferido por diff), medido no Neon do aluno:

Versão	Tempo mediano
v1 (original, junta antes de agrupar)	734,4 ms
v2 (agrega antes de juntar)	486,4 ms

Ganho de 33,8% sem criar nenhum índice novo — puro reordenamento da lógica da consulta.

A4 — Contraprova de seletividade

Criamos idx_pedido_canal_ignorado sobre pedido.canal_venda, que tem só 3 valores (aplicativo ~60%, site ~25%, whatsapp ~15%). Testando com um filtro ainda menos seletivo (status IN ('pago','enviado','entregue'), ~82% da tabela), o Postgres optou por Seq Scan mesmo com o índice disponível — confirmado no Neon do aluno:

```
Seq Scan on pedido  (actual time=0.012..106.990 rows=492273)   Filter: status = ANY ('{pago,enviado,entregue}')   Rows Removed by Filter: 107727   Execution Time: 129.944 ms
```

Seletividade = linhas que passam no filtro ÷ linhas da tabela (82% aqui); acima de ~15–20% o Index/Bitmap Scan deixa de compensar o Seq Scan sequencial.

A5 — Custo de escrita

Nota de transparência: tentamos medir A5 diretamente no banco de produção do aluno (Neon), mas o plano gratuito atingiu seu limite de armazenamento (512 MB) mesmo para operações mínimas (uma CREATE TABLE vazia, um índice de uma coluna numa tabela de 51 linhas). Tentamos 6 contornos diferentes — reduzir o volume do teste, reduzir a janela de retenção de histórico do Neon (de 6h para 0h), reiniciar o compute, e reaproveitar o espaço de um índice já existente — sendo que o último funcionou uma vez (permitiu concluir a A6) mas não se repetiu para a A5. Documentamos essa limitação como parte do processo, e usamos os números do ambiente de validação (mesma estrutura de tabela, mesmo volume completo do contrato) para a comparação abaixo.

Cenário	Tempo de inserir 50.000 pedidos
(i) Sem índices (só chave primária)	82,0 ms
(ii) Com 3 índices (status+data, cliente+data, uf_entrega)	325,1 ms

Inserir com os 3 índices foi ~3,96x mais lento — cada índice precisa ser atualizado a cada linha inserida. Quantos índices vale a pena manter depende do perfil de leitura vs escrita da tabela: pedido é escrita uma vez por pedido novo mas lida centenas de vezes ao dia em consultas analíticas, então vale manter os índices mesmo com esse custo.

A6 — Índice redundante

Criamos idx_pedido_status_data (status, data_pedido), que é PREFIXO de idx_pedido_status_data_uf (status, data_pedido, uf_entrega) — qualquer consulta que usa o primeiro também poderia usar o segundo. Medido no Neon do aluno:

Índice	Tamanho
idx_pedido_status_data_uf (3 colunas, mantido)	25 MB
idx_pedido_status_data (2 colunas, redundante)	20 MB

Depois de remover o índice de 2 colunas, uma consulta que filtra só por status+data (padrão de Q1/Q2/Q4/Q8) continuou usando idx_pedido_status_data_uf via Index Only Scan, sem nenhuma mudança de custo — prova de que era redundante. Os 20 MB foram recuperados. Por isso o índice de 2 colunas NÃO aparece na versão final de sql/indices.sql.

Achado extra (não pedido, mas relevante)

Durante os testes no ambiente de validação, um DELETE de 50.000 pedidos travou por mais de um minuto. O motivo: pagamento.pedido_id é uma FK sem índice — a cada linha apagada de pedido, o Postgres precisa varrer pagamento (492 mil linhas) inteira para checar integridade referencial. Criamos idx_pagamento_pedido e o mesmo DELETE passou a rodar em menos de 1 segundo. Ilustra que FK sem índice pode inviabilizar manutenção, não só leitura.

6. Pergunta final — qual foi o ganho do conjunto?

Considerando as 10 consultas medidas (9 de negócio + legada) no ambiente do aluno, o ganho variou de -1,7% (Q8, praticamente neutro) a 90,7% (Q5), com mediana em torno de 8-9%. A técnica que mais rendeu, em termos absolutos, foi índice em filtro seletivo (Q5: 90,7%; legada: 67,9%) — não porque índice seja sempre a melhor técnica, mas porque essas duas consultas tinham a característica certa para ele (filtro que elimina a maior parte da tabela). A reescrita sem índice (A3, ganho de 33,8% na Q7) também rendeu de forma consistente e sem custo de escrita ou espaço extra — vale sempre considerar antes de criar um índice novo. Atualizar estatística (ANALYZE) não pôde ser isolado como um número de "ganho" nesta bateria porque rodamos ANALYZE antes de toda medição (a linha de base já incluía estatística em dia, como o protocolo pede); seu efeito aparece indiretamente em toda decisão de plano que depende de estimativa de seletividade correta, como a escolha de Seq Scan na A4.

Se o sistema tivesse 50x mais pedidos (30 milhões): esperaríamos que (a) os ganhos de Q5/legada/A3 se mantivessem ou aumentassem, porque a vantagem de um índice seletivo cresce com o tamanho da tabela (Seq Scan piora linearmente, Index Scan quase não piora); (b) Q1/Q2/Q3/Q8, que já são gargalo por agregarem boa parte da tabela, ficariam praticamente inviáveis do jeito que estão — a saída não seria mais índice, e sim pré-agregação incremental (uma tabela de fatos ou materialized view atualizada por trigger ou job, por exemplo totais diários por categoria); e (c) particionaríamos pedido e item_pedido por mês (data_pedido), já que quase todas as consultas filtram por período — partition pruning eliminaria meses inteiros do plano antes mesmo de abrir um índice.

7. Limitações

- O plano gratuito do Neon impôs um limite de armazenamento (512 MB) que impediu a medição ao vivo da Análise A5 no ambiente de produção do aluno — documentado em detalhe na seção A5, incluindo as 6 tentativas de contorno realizadas.
- Durante as medições, notamos que a contagem de blocos lidos cresce a cada execução sucessiva da mesma consulta no Neon (diferente do comportamento estável observado num Postgres local) — provavelmente por a camada de armazenamento serverless do Neon reconstruir cache de forma diferente a cada conexão. Por isso demos mais peso ao tempo mediano do que aos blocos como métrica de comparação nesta bateria específica.
- O gerador de dados usa distribuições simplificadas (uniformes/pesos fixos) — reproduz as proporções do contrato, mas não modela correlações finas do mundo real (ex.: cliente que sempre paga no mesmo método).
- O banco (~600 mil pedidos, ~1,6 milhão de itens) provavelmente cabe majoritariamente em cache no ambiente de teste — o próprio enunciado avisa que isso pode achatar a diferença entre planos distintos, e é um motivo plausível para os ganhos pequenos de Q1/Q3/Q8.